Dragon Ball
Catálogo Z

Desenvolvimento Mobile Híbrido
Bacharelado em Sistemas de Informação (BSI)
Professor: Mark Joselli

Turma: PREENCHER
Data de entrega: 02/11/2026

Integrantes

PREENCHER NOME COMPLETO DE TODOS OS INTEGRANTES

RASCUNHO: faltam dados do grupo e link do vídeo público. Capturas obtidas na versão web em viewport móvel. Supabase validado com duas contas de teste.

1. Links

Repositório GitHub público: https://github.com/ODevAbreu/CatalogoDragonBall

Vídeo YouTube público: PREENCHER LINK PUBLICO DO YOUTUBE

2. Tema e configuração

Tema: personagens de Dragon Ball. API: https://dragonball-api.com/api. Framework: React Native com Expo e TypeScript. Tema do somativo Flutter: Pokémon, portanto diferente deste projeto.

Persistência obrigatória: AsyncStorage. Login local: PBKDF2-HMAC-SHA256 com salt e sessão em SecureStore no celular. Itens consumidos são chamados de conhecidos, com avaliação de 1 a 5 estrelas.

Extras implementados e validados: Supabase Auth e coleção em dragonball_items no mesmo projeto Supabase do Flutter. A tabela Pokémon user_items permanece separada. Teste real confirmou cadastro/login, gravação e recuperação da coleção, renovação de sessão, desmarcações e isolamento RLS entre duas contas QA.

3. Autoavaliação RF01-RF10

RF	Implementado	Arquivo principal
RF01	Sim	src/hooks/useCharacters.ts src/components/CharacterGrid.tsx
RF02	Sim	src/navigation/RootNavigator.tsx
RF03	Sim	src/screens/DetailScreen.tsx
RF04	Sim	src/contexts/CollectionContext.tsx
RF05	Sim	src/screens/CollectionScreen.tsx
RF06	Sim (local + nuvem)	src/contexts/CollectionContext.tsx src/services/storage.ts
RF07	Sim (local + Auth real)	src/services/auth.ts src/contexts/AuthContext.tsx
RF08	Sim	src/screens/SearchScreen.tsx
RF09	Sim	src/components/ui.tsx src/hooks/useCharacters.ts
RF10	Parcial: validar leitor nativo	src/components/ src/theme/index.ts

Autoavaliação baseada no código, testes web e teste real do Supabase. RF10 tem labels, roles, estados e adaptação de fonte implementados; TalkBack/VoiceOver em aparelho ainda deve ser validado. Bônus RF06/RF07: autenticação, persistência e RLS validados no servidor.

4. Flutter × React Native

A Pokédex Flutter separava serviços, providers e widgets; o Catálogo Z preserva as responsabilidades usando serviços, Context e componentes. ChangeNotifier e notifyListeners dão lugar a createContext/useContext e estados React. Favoritos e sessão são globais; o texto da busca fica em useState local, evitando espalhar atualizações que só interessam à tela de busca.

GridView dá lugar à FlatList virtualizada, com numColumns, keyExtractor e rolagem infinita. Em vez do botão de carregar mais do Flutter, onEndReached busca a página seguinte. Uma falha preserva os itens e oferece nova tentativa no rodapé. Ao contrário da busca Flutter que abria diretamente um detalhe, a busca React Native mostra uma lista de resultados.

4. Comparação (continuação)

FutureBuilder e providers assíncronos foram adaptados para useEffect/useState. O cleanup usa AbortController para cancelar consultas antigas. Gravações locais são serializadas e a sincronização remota reconcilia alterações pela data. O gate inicial aguarda recuperar a sessão e a coleção, evitando mostrar dados zerados.

Hot reload e ferramentas Dart são comparáveis ao Fast Refresh e ao fluxo de QR code do Expo. TypeScript ajuda a verificar modelos e rotas antes de executar. A produtividade percebida deve ser complementada com a experiência real dos integrantes. Testes de navegador não substituem validação em Android/iOS.

5. Tema da explicação técnica

Tema: Context API, estado local e persistência. Minuto: 5:30 (confirmar no video final)

Mostrar o caminho UI -> CollectionContext -> AsyncStorage -> Supabase. Explicar por que favoritos e sessão são globais, como useEffect restaura a coleção e como o logout altera a árvore de navegação. Comparar com Provider do Flutter e discutir renderizações dos consumidores. Roteiro completo em docs/roteiro-video.md, duração alvo de 8 minutos.

Evidências e limites da verificação

13 testes unitários aprovados: contrato paginado/filtrado, busca, falhas, HTML, conflitos/fusos, desmarcações e notas. Fluxo web aprovado com cadastro, login, paginação com falha e recuperação, favoritos, conhecido, nota, busca, reabertura, logout e isolamento de contas. TypeScript, lint, compatibilidade Expo e bundles web/Android verificados. Integração Supabase testada com duas contas QA.

As próximas páginas apresentam screenshots reais da interface web em viewport 390 × 844. Dados da API foram reproduzidos por fixtures no teste e imagens vêm dos URLs públicos originais. Não representam evidência de autenticação remota, leitor de tela nativo ou execução Android.

Pendências antes da submissão

Preencher integrantes, turma e link do vídeo. Executar no dispositivo, testar leitor de tela e fonte ampliada. Gravar todos os integrantes falando. Publicar o código no repositório indicado quando autorizado, revisar o PDF final e enviar pelo campo oficial do AVA.

6. Screenshots das telas principais

Login e cadastro



Catálogo paginado



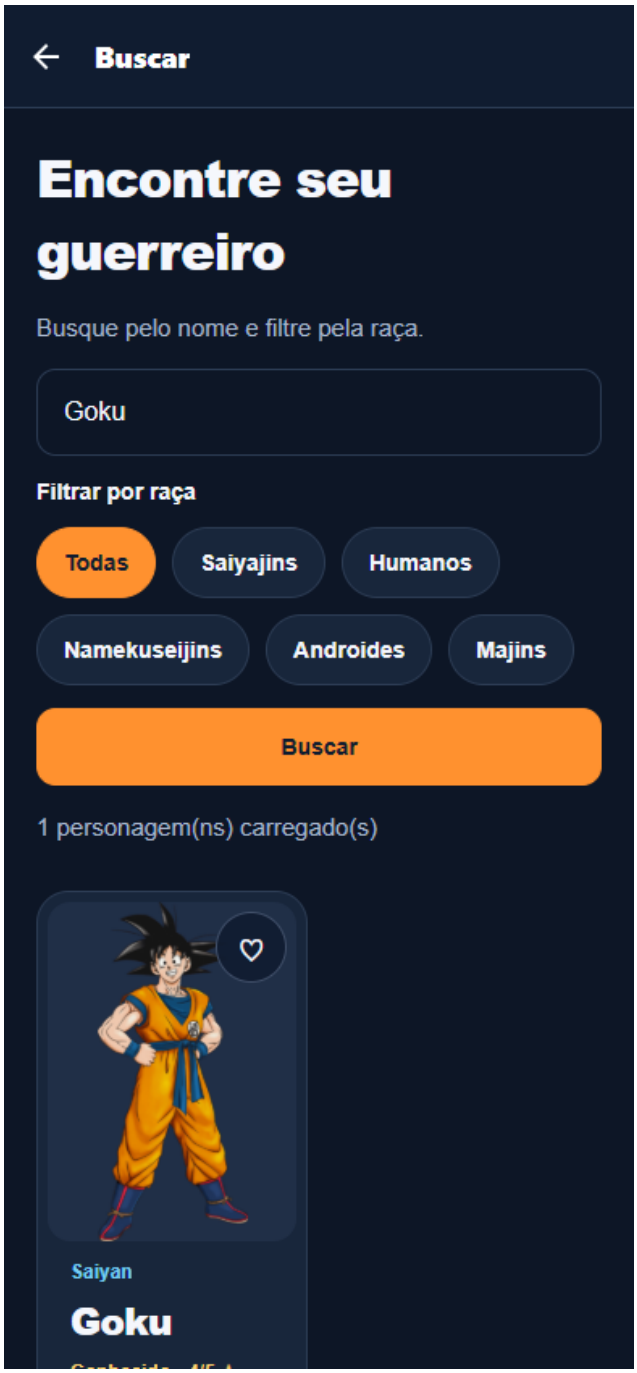
Execução web em viewport móvel. Screenshots geradas pelo teste automatizado.

6. Screenshots das telas principais

Detalhes, favorito e nota



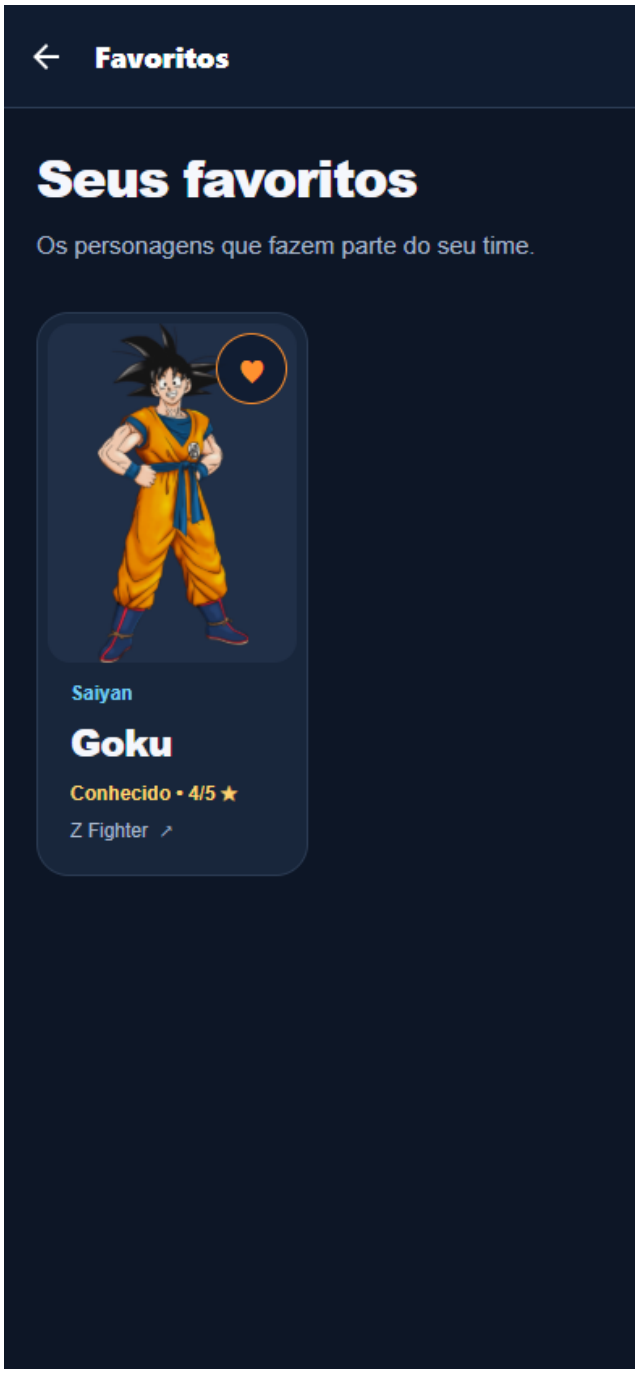
Busca por personagem



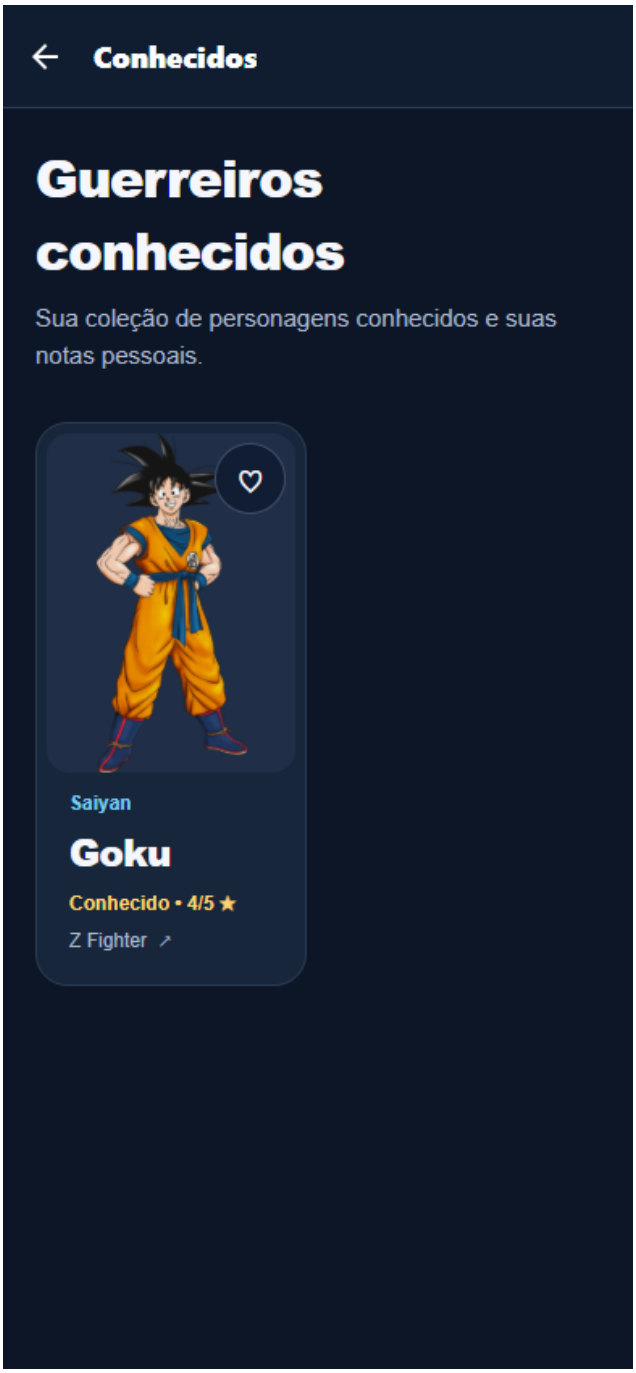
Execução web em viewport móvel. Screenshots geradas pelo teste automatizado.

6. Screenshots das telas principais

Favoritos



Conhecidos e avaliações



Execução web em viewport móvel. Screenshots geradas pelo teste automatizado.